

Rayls Network Node – Consensus Protocol

RAYLSFUSION LTD. (ParFin)

HALBORN

Rayls Network Node – Consensus Protocol - RAYLSFUSION LTD. (ParFin)

Prepared by:  HALBORN

Last Updated 04/20/2026

Date of Engagement: March 2nd, 2026 - March 20th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
5	1	0	3	1	0

TABLE OF CONTENTS

- 1. Introduction
- 2. Assessment summary
- 3. Test approach and methodology
- 4. Risk methodology
- 5. Scope
- 6. Assessment summary & findings overview
- 7. Findings & Tech Details
 - 7.1 Insufficient validation in parallel certificate fetching enables persistent database poisoning
 - 7.2 Consensus quorum falsification via validator double-voting
 - 7.3 Unauthenticated manipulation of epoch synchronization state allows permanent denial of service
 - 7.4 Double-voting protection bypass due to race condition in request handler
 - 7.5 Plaintext bls private key storage is allowed

1. Introduction

ParFin engaged Halborn to conduct a security assessment on their smart contracts and consensus system from March 2nd, 2026 to March 20th, 2026. The security assessment scope was limited to the **Rayls Network** node consensus implementation and associated on-chain behaviors.

The assessment targeted consensus-related Rust code paths, on-chain verification logic, certificate synchronization, gossip handling, and validator key management. The purpose of this engagement was to identify security vulnerabilities that could compromise consensus safety, liveness, or validator key confidentiality, and to recommend mitigations to preserve network integrity.

2. Assessment Summary

Halborn was allocated 19 days for this engagement and assigned one full time security engineer to conduct a comprehensive review of the consensus mechanisms within scope. The security engineer is a blockchain and Rust security expert with advanced knowledge of consensus protocols, distributed systems, peer-to-peer networking, state machine execution, and denial-of-service resilience. The assessment was executed specifically against Rayls Network node consensus components.

The purpose of this assessment is to:

- Ensure that the node correctly enforces protocol invariants in consensus module.
- Identify potential vulnerabilities related to consensus components.

In summary, Halborn identified several recommendations to strengthen the node's resilience and operational safety which were completely addressed by **ParFin** team. The most relevant ones are the following:

- Enforce direct and full cryptographic verification of BLS aggregate signatures on all ancestor certificates to prevent persistent database poisoning during state synchronization.
- Track individual validator identities within the consensus result aggregation map to prevent quorum falsification resulting from repeated double voting.
- Require mathematically verifiable authentication of gossip messages before allowing any updates to the internal epoch synchronization state to prevent denial of service.
- Resolve the memory cache race condition in the request handler by maintaining state locks during asynchronous database writes to properly enforce equivocation protections.
- Mandate the use of encryption for validator BLS private key storage and completely remove the fallback path that allows plaintext saving when a passphrase is omitted.

3. Test Approach And Methodology

A layered testing approach was employed consisting of code review, targeted manual analysis of consensus and gossip logic, and automated scanning where applicable. The sequence of phases, as reflected in the supplied findings, was:

- Research and scoping to identify critical on-chain and node-level attack surfaces (certificate handling, gossip handler, vote processing, key storage).
- Manual review of Rust source code paths implementing certificate verification, gossip processing, consensus result aggregation, vote caching, and key generation/storage.
- Targeted proof-of-concept reasoning and code-level reproduction snippets for high-risk classes of issues (e.g., indirect signature verification bypass, `consensus_certs` aggregation logic, `requested_missing_epoch` manipulation, equivocation race).
- Automated analysis and static tooling where applicable to flag patterns and low-hanging implementation weaknesses (details in Automated Testing below).
- Prioritization of findings by exploitability and impact to consensus safety and liveness, with remediation guidance provided for each item.

The balance favored deep manual review for consensus-critical logic, since cryptographic and state-machine errors were considered high-impact and often not reliably detected by automated scanners. The methodology aimed to ensure thorough coverage of trust boundaries, cross-module interactions, and persistence semantics without producing excessive false positives.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

FILE

(a) Submitted File: axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11.zip

(b) Items in scope:

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.github/workflows/pr.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.github/ACTIONS.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.vscode/launch.json.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.vscode/tasks.json.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/bin/rayls-network/src/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/bin/rayls-network/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/mainnet/committee.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/mainnet/genesis.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/mainnet/parameters.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/testnet/committee.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/testnet/genesis.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/testnet/parameters.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/testnet/testnet_accounts.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/chain-configs/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/src/tests/metered_channel_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/src/guards.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/src/histogram.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/src/metered_channel.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/consensus-metrics/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/all_peers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/banne

d.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/behavior.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/cache.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/manager.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/mod.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/peer.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/README.md

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/score.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/status.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/peers/types.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/banned_peers.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/cache_peers.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/codec_tests.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/common.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/network_tests.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/peer_manager.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/peers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/tests/types.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/codec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/kad.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/src/types.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/network/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/aggregators/certificates.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/aggregators/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/aggregators/votes.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/tests/bullshark_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/tests/consensus_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/tests/leader_schedule_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/tests/randomized_tests.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/bullshark.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/leader_schedule.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/state.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus/utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/error/cert_manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/error/gc.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/error/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/error/network.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/error/proposer.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/network/handler.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/network/message.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/network/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/cert_collector.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/cert_manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/cert_validator.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/gc.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/header_validator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_sync/pending_cert_manager.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/test_utils/helpers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/test_utils/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/cert_collector_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/cert_manager_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/cert_validator_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/certificate_fetcher_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/certificate_processing_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/certificate_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/certifier_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/header_validator_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/network_tests.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/primary_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/proposer_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/tests/synchronizer_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/certificate_etcher.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/certifier.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/consensus_bus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/primary.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/proposer.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/recent_blocks.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/src/state_handler.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/tests/it/certificate_order_test.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/tests/it/output_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/tests/it/storage_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary-metrics/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary-metrics/src/executor_metrics.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary-metrics/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary-metrics/src/metrics.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/primary-metrics/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/epoch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/state-sync/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/src/test_utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/tests/it/build_batches.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-builder/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/src/validator.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch-validator/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/handler.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/message.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/network/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/batch_provider_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/network_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/it/quorum_waiter_tests.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/tests/batch_fetcher.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/batch_fetcher.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/metrics.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/quorum_waiter.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/test_utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/src/worker.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/consensus/worker/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/benches/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/block.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/config.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/context.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/factory.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/handler.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/evm/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/abi.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/composite_inspector.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/inspector.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/precompile.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/native_erc20/storage.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/bypass_validator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/chainspec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/dirs.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/payload.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/persistence.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/rpc_server_args.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/system_calls.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/test_utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/traits.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/txn_pool.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/src/worker.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/evm/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/src/cli_ext.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/src/rpc_ext.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/src/service.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/tests/it/main.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/faucet/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/rpc/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/rpc/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/rpc/src/rpc_ext.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/execution/rpc/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/genesis.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/keys.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/network.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/node.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/retry.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/src/traits.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/config/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/args/consensus_database.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/args/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/args/utils.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/genesis/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/keytool/generate.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/keytool/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/keytool/stake_calldata.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/cli.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/node.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/src/version.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/build.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-cli/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-types/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-types/src/local.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-types/src/notify.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-types/src/response.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/network-types/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/dat

abase.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/metrics.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mdbx/mod.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/database.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/metrics.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/mod.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/redb/wraps.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/certificate_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/checkpoint_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/consensus_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/epoch_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/mod.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/payload_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/proposer_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/stores/vote_digest_store.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/layered_db.rs

- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/lib.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/src/mem_db.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/storage/Cargo.toml
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_keypair.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_public_key.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/bls_signature.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/intent.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/mod.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/crypto/network.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/block.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/certificate.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/epoch.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/header.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/info.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/mod.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/output.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/reputation.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/primary/vote.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/test_utils/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/test_utils/racing.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/mod.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/pending_batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/worker/sealed_batch.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/batch_tracker.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/codec.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/committee.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/database_traits.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/error.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/gas_accumulator.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/genesis.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/helpers.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/notifier.rs

- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/serde.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/sync.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/src/task_manager.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/types/Cargo.toml
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/utils/src/lib.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/utils/src/notify_read.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/infrastructure/utils/Cargo.toml
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/errors.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/lib.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/src/subscriber.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/tests/it/main.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/Cargo.toml
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/bridge/README.md
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/builder.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/inner.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/engine/mod.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/tests/epoch_transition_tests.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/epoch_transition.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/error.rs
- /axyl-
068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/health.

rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/lib.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/manager.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/primary.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/src/worker.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/tests/it/main.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/Cargo.toml

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/orchestrator/README.md

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/error.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/lib.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/src/payload_builder.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/tests/it/main.rs

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/Cargo.toml

- /axyl-

068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/middleware/processor/README.md

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/src/lib.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/tests/it/epochs.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/tests/it/faucet.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/tests/it/genesis_tests.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/tests/it/main.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/tests/it/restarts.rs

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/e2e-tests/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils/src/consensus.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils/src/execution.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils/src/temp_dirs.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/src/authority.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/src/builder.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/src/committee.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/src/lib.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/src/worker.rs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/crates/testing/test-utils-committee/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/debug-network/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/debug-network/init.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/debug-network/launch_validators.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/debug-network/relaunch.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/debug-network/setup-for-launch.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/docker-network/compose.yaml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/docker-network/genesis.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/docker-network/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/docker-network/setup_validator.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/config/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/config/topology.json.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/client-node/client-nodes.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/client-node/genesis.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/client-node/start.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/host-node/docker-host-nodes.sh

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/host-node/host-nodes.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/utils/init.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/utils/ssh.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/utils/topology.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/validate/host-nodes.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/validate/script-requirements.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/src/validate/topology.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/manage.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/manage/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/observer/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/observer/create-observer.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/observer/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test/test-and-attest.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/clear-validators-db.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/erc20_precompile_test.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/local-testnet.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/start-local-validator-1.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/start-local-validator-2.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/start-local-validator-3.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/test-network/start-local-validator-4.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/config/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/blockchain/utils.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/config/config.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/monitor/monitor.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/tx/encoder-thread.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/tx/encoder.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/tx/sender.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/utils/utils.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/src/main.mjs
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/check-tps.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/package-lock.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/package.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/tps/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/validator/.env.example

- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/validator/create-validator.sh](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/validator/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/etc/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/github/workflows/test.yml](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/AxelarAmplifierGateway.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/BlsG1.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/ConsensusRegistry.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/ERC1967Proxy.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/Issuance.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/Stablecoin.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/artifacts/StablecoinManager.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/genesis/GenesisPrecompiler.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/genesis/precompile-config.yaml](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/genesis/R1Genesis.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/utills/Create3Utils.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/utills/ITSConfig.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/utills/ITSUtils.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/deployments.json](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/deployments/Deployments.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/axelar-cgp-solidity/AxelarGasServiceProxy.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/rayls-contracts/interfaces/IStablecoin.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/interfaces/IUniswapV2Factory.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/interfaces/IUniswapV2Pair.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/interfaces/IUniswapV2Router01.sol](#)

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/interfaces/IUniswapV2Router02.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/precompiles/UniswapV2FactoryBytecode.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/external/uniswap/precompiles/UniswapV2Router02Bytecode.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/approve.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/execute.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/initiate.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/proof.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/relay-instructions.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/route.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/devnet/verify.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/includer.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/latest-task.txt
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/subscriber.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/relay/utils.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/node/src/verifier-instructions.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/bash/fund-pool-add-rewards.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/bash/instantiate-internal-gateway.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/bash/instantiate-multisig-prover.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/bash/instantiate-voting-verifier.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/bash/query-rewards-pool-balance.sh
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/deploy/TestnetDeployGitAttestationRegistry.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/deploy/TestnetDeployStablecoinManager.s.sol

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/deploy/TestnetDeployTokens.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/deploy/TestnetDeployUniswapV2.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/TestnetFundDeveloper.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/TestnetGrantRole.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/TestnetManageFaucet.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/testnet/TestnetUpgradeStablecoinManager.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/GeneratePrecompileGenesisConfig.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/InterchainTransfer.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/script/LinkToken.s.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/CI/GitAttestationRegistry.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/BlsG1.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/ConsensusRegistry.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/design.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/invariants.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/Issuance.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/StakeManager.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/consensus/SystemCallable.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/faucet/RLFaucet.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/faucet/StablecoinManager.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/interfaces/IConsensusRegistry.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/interfaces/IInterchainRLS.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/interfaces/IRecoverableWrapper.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/interfaces/IStakeManager.sol

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/recoverable-wrapper/RecordUtil.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/recoverable-wrapper/RecoverableWrapper.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/design.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/InterchainRLS.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/its-design.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/src/WRLS.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/CI/GitAttestationRegistry.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/consensus/ConsensusRegistryTest.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/consensus/ConsensusRegistryTestFuzz.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/consensus/ConsensusRegistryTestUtils.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/EIP2537/BlsG1.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/EIP2537/BlsG1Harness.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/faucet/StablecoinManagert.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/ITS/InterchainRLSForkTest.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/ITS/InterchainRLSTest.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/ITS/InterchainTokenServiceForkTest.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/ITS/InterchainTokenServiceTest.t.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/test/ITS/ITSTestHelper.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/.gitignore
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/foundry.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/package-lock.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/package.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/remappings.txt
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts/tsconfig.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/interfaces/IERC20Withdrawable.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/interfaces/IP0SBridge.sol
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/test/MockBridgeRelay.sol

- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/test/TestPOSBridge.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/test/TestPredicate.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/BridgeRelay.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/bridge/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/external/openzeppelin/test/MockClonableBeaconProxy.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/external/openzeppelin/ClonableBeaconProxy.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/factories/library/FactoryStorage.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/factories/ProxyFactory.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/factories/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/stablecoin/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/stablecoin/Stablecoin.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/stablecoin/StablecoinHandler.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/interfaces/ISimplePlugin.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/test/MockAmirX.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/test/TestAggregator.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/test/TestPlugin.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/AmirX.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/swap/README.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/test/TestToken.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/test/TestWallet.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/contracts/util/abstract/Blacklist.sol](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/docs/index.md](#)
- [/axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/docs/Sherlock_Audit.pdf](#)

- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/scripts/deploy.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/bridge/BridgeRelay.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/external/ClonableBeaconProxy.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/factories/ProxyFactory.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/stablecoins/Stablecoin.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/stablecoins/StablecoinHandler.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/swap/AmirX.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/test/util/Blacklist.test.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/.gitignore
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/.npmrc
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/hardhat.config.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/index.ts
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/package.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rayls-contracts-npm/tsconfig.json
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.env.example
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/.gitignore
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/Cargo.lock
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/Cargo.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/clippy.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/CONTRIBUTING.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/github.pub
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/README.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/rustfmt.toml
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/SECURITY.md
- /axyl-068cb4a881621b9df570ff99a1f77e1e81b34d11/SYNC.md

REMEDATION COMMIT ID:



- <https://github.com/raylsnetwork/axyl/pull/136>
- <https://github.com/raylsnetwork/axyl/pull/266>
- <https://github.com/raylsnetwork/axyl/pull/138>
- <https://github.com/raylsnetwork/axyl/pull/267>
- <https://github.com/raylsnetwork/axyl/pull/272>

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW



SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
INSUFFICIENT VALIDATION IN PARALLEL CERTIFICATE FETCHING ENABLES PERSISTENT DATABASE POISONING	CRITICAL	SOLVED - 03/11/2026
CONSENSUS QUORUM FALSIFICATION VIA VALIDATOR DOUBLE-VOTING	MEDIUM	SOLVED - 03/24/2026
UNAUTHENTICATED MANIPULATION OF EPOCH SYNCHRONIZATION STATE ALLOWS PERMANENT DENIAL OF SERVICE	MEDIUM	SOLVED - 03/10/2026
DOUBLE-VOTING PROTECTION BYPASS DUE TO RACE CONDITION IN REQUEST HANDLER	MEDIUM	SOLVED - 03/24/2026
PLAINTEXT BLS PRIVATE KEY STORAGE IS ALLOWED	LOW	SOLVED - 03/26/2026

7. FINDINGS & TECH DETAILS

7.1 INSUFFICIENT VALIDATION IN PARALLEL CERTIFICATE FETCHING ENABLES PERSISTENT DATABASE POISONING

// CRITICAL

Description

The consensus primary node contains a critical vulnerability in how it validates batches of certificates fetched from peers during state synchronization. When a node fetches missing certificates, it attempts to optimize cryptographic verification by only performing full BLS aggregate signature verification on leaf certificates and skipping it for any certificate that is claimed as a parent by another certificate in the fetched batch.

The underlying assumption is that if a child certificate is validly signed by the committee, its parents must also be valid. However, while the digest of the parent is authenticated by the child's signature, the signature (`SignatureVerificationState`) and `signed_authorities` bitmap of the parent are not part of the digest.

An attacker can exploit this by fetching a legitimately signed **CertificateDAG** from the network, replacing the `BlsAggregateSignature` of all ancestor certificates with empty or arbitrary values, and sending the batch alongside a valid leaf certificate during a P2P state sync.

The honest victim node will verify the leaf child, see that the parent's digest matches the child's list of parents, and inappropriately mark the parent as `VerifiedIndirectly` with the manipulated signature, bypassing all cryptographic verification for the ancestor.

The victim node then writes the corrupted parent certificate into its persistent `CertificateStore`. This creates a permanent chain desync and an Eclipse Denial of Service (DoS) attack, as the victim's local database is permanently poisoned with invalid cryptographic data. When the victim attempts to use this certificate or serve it to other honest nodes, the verification will fail because the signature is invalid, causing peers to reject the data and potentially penalize or disconnect the victim node. Because the corrupted certificate possesses the mathematically correct digest, the poisoned node will never identify it as missing and will never attempt to re-fetch the correct version, permanently isolating the node from the consensus network.

Code Location

Code of `requires_direct_verification` function from `consensus/primary/src/state_sync/cert_validator.rs` file:

 Copy Code

```
fn requires_direct_verification(
    &self,
    cert: &Certificate,
    all_parents: &HashSet<CertificateDigest>,
) -> bool {
```

```

!all_parents.contains(&cert.digest())
}
fn mark_verified_indirectly(&self, cert: &mut Certificate) -> CertManagerResult<()> {
    cert.set_signature_verification_state(SignatureVerificationState::VerifiedIndirectly(
        cert.aggregated_signature()
            .ok_or(CertificateError::RecoverBlsAggregateSignatureBytes)?,
    ));
    Ok(())
}

```

Proof of Concept

Scenario

The vulnerability can be easily reproduced using the protocol's own test suite. By adding the following unit test to `crates/consensus/primary/src/tests/cert_validator_tests.rs`, we demonstrate that an attacker can successfully inject completely invalid signatures (`BlsSignature::default()`) into an honest validator's processing pipeline.

Test Code

 Copy Code

```

#[tokio::test]
async fn test_poc_v009_signature_bypass() -> eyre::Result<()> {
    let TestTypes { validator, cb, fixture, task_manager: _task_manager } = create_test_types();

    let committee = fixture.committee();
    let genesis_certs = Certificate::genesis(&committee);
    let genesis = genesis_certs.iter().map(|x| x.digest()).collect::<BTreeSet<_>>();
    let keys = fixture.authorities().map(|a| (a.id(), a.keypair().copy())).collect::<Vec<_>>();

    let (certificates, _) = make_optimal_signed_certificates(1..=60, &genesis, &committee, &keys);
    let unverified_certificates = certificates.into_iter().collect::<Vec<_>>();

    let mut poisoned_batch = Vec::new();
    for cert in &unverified_certificates {
        let mut poisoned_cert = cert.clone();
        if cert.round() != 50 && cert.round() != 60 {
            poisoned_cert.set_signature_verification_state(
                SignatureVerificationState::Unverified(BlsSignature::default())
            );
        }
        poisoned_batch.push(poisoned_cert);
    }

    let mut certificate_manager_rx = cb.certificate_manager().subscribe();
    let expected_count = poisoned_batch.len();

    let h_task = tokio::task::spawn(async move {
        let mut count = 0;
        while count < expected_count {
            if let Some(CertificateManagerCommand::ProcessVerifiedCertificates { certificates, reply }) = certificate_manager_rx.recv().await {
                let _ = reply.send(Ok(()));
                count += certificates.len();
            }
        }
    });

    let result = validator.process_fetched_certificates_in_parallel(poisoned_batch).await;
    assert!(result.is_ok(), "Vulnerability: Falsified signatures were accepted");
    h_task.await?;

    Ok(())
}

```

Result

 Copy Code

```
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.58s
Running unittests src/lib.rs (target/debug/deps/rayls_consensus_primary-80ab01dc4b24ea66)

running 1 test
test state_sync::cert_validator::cert_validator_tests::test_poc_v009_signature_bypass ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 67 filtered out; finished in 5.20s

Running tests/it/main.rs (target/debug/deps/it-3c200c95e17371a8)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 16 filtered out; finished in 0.00s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:H/D:N/Y:N (9.4)

Recommendation

It is recommended to remove the `VerifiedIndirectly` optimization from the parallel certificate fetcher. All certificates, regardless of their position in the Direct Acyclic Graph and whether their digest is authenticated by a child, must possess valid BLS aggregate signatures from the committee and undergo direct cryptographic validation before being accepted and persisted to the database.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by removing the `VerifiedIndirectly` optimization entirely. All certificates in every fetched batch are now unconditionally routed through `verify_certificate_chunk` for full BLS aggregate signature verification before being accepted and persisted to the database.

Remediation Hash

<https://github.com/raylsnetwork/axyl/pull/136>

7.2 CONSENSUS QUORUM FALSIFICATION VIA VALIDATOR DOUBLE-VOTING

// MEDIUM

Description

The `RequestHandler::process_gossip` function in `handler.rs` fails to track individual validator votes when aggregating consensus results, allowing a single malicious validator to falsify a consensus quorum.

When a `PrimaryGossip::Consensus(result)` message is received, the handler extracts the `ConsensusResult`, which contains an epoch, round, number, hash, validator (public key), and signature. The handler calculates a `consensus_result_hash` using `result.digest()`. Crucially, `ConsensusResult::digest()` only includes the epoch, round, number, and hash fields; it does not include the validator key or the signature.

The handler verifies the signature against the provided validator key but then proceeds to increment a simple counter in a `HashMap<BlockHash, u32>` called `consensus_certs`. Because this map only stores the total number of valid signatures seen for a given result hash and does not track which validators have already contributed, a single validator can send the same (or different) signed messages multiple times. Each message will pass the signature verification and increment the counter. Once the counter reaches the threshold `enough_sigs` (typically $1/3 + 1$ of the committee), the node marks the consensus result as valid and publishes it to the `consensus_bus`, potentially leading to the finalization of an arbitrary or malicious block hash.

Code Location

Code of `process_gossip` function from `crates/consensus/primary/src/network/handler.rs` file:

 Copy Code

```
let sigs = self.consensus_certs.lock().get(&consensus_result_hash).copied();
if let Some(sigs) = sigs {
    if (sigs + 1) as usize >= enough_sigs {
        if self.behind_consensus(epoch, round, Some(number)).await {
            self.consensus_certs.lock().clear();
            return Ok(());
        }
        self.consensus_bus
            .last_published_consensus_num_hash()
            .send_replace((number, hash));
        self.consensus_certs.lock().clear();
    } else {
        self.consensus_certs.lock().insert(consensus_result_hash, sigs + 1);
    }
} else {
    self.cleanup_consensus_certs();
    self.consensus_certs.lock().insert(consensus_result_hash, 1);
}
```

Code of `digest` and `digest_data` function from `crates/consensus/primary/src/network/messages.rs` file:

```
pub fn digest(&self) -> BlockHash {
    Self::digest_data(self.epoch, self.round, self.number, self.hash)
}

pub fn digest_data(epoch: Epoch, round: Round, number: u64, hash: BlockHash) -> BlockHash {
    let mut hasher = DefaultHashFunction::new();
    hasher.update(&epoch.to_be_bytes());
    hasher.update(&round.to_be_bytes());
    hasher.update(&number.to_be_bytes());
    hasher.update(hash.as_ref());
    B256::from_slice(hasher.finalize().as_bytes())
}
```

BVSS

AO:A/AC:L/AX:L/R:P/S:C/C:N/A:H/I:H/D:N/Y:N (5.9)

Recommendation

It is recommended to modify the consensus_certs data structure to track the set of validator keys that have contributed to each consensus result hash. Instead of a simple u32 counter, a HashSet or a bitmask (if the committee members are indexed) should be used. The handler must verify that a validator hasn't already contributed a vote for the same result hash before updating the quorum state. Additionally, the system should consider implementing stricter replay protection at the gossip layer to prevent the same signed message from being processed multiple times.

Remediation Comment

SOLVED: The ParFin team solved this issue by replacing the plain `HashMap<BlockHash, u32>` counter in `consensus_certs` with a generic `VotesAggregator<ConsensusResult>` that tracks seen signers via a `HashSet<AuthorityIdentifier>`, so duplicate gossip messages from the same validator for the same consensus result digest no longer inflate the quorum count.

Remediation Hash

<https://github.com/raylsnetwork/axyl/pull/266>

7.3 UNAUTHENTICATED MANIPULATION OF EPOCH SYNCHRONIZATION STATE ALLOWS PERMANENT DENIAL OF SERVICE

// MEDIUM

Description

The primary consensus network handler processes incoming `PrimaryGossip::Consensus` messages to track the progression of the network. When a gossip message references an epoch for which the local node does not possess the committee keys, the node is unable to mathematically verify the associated signature. Instead of dropping the unauthenticated message, the handler extracts the assumed `epoch` value from the message payload and uses it to overwrite the internal `requested_missing_epoch` state, provided the newly received value is strictly greater than the current one.

Because this state update occurs unconditionally for unknown committees, any arbitrary peer can broadcast a crafted consensus result encapsulating the maximum possible epoch value, regardless of whether they possess legitimate validator credentials. Upon receiving this message, the target node persists the artificially inflated epoch value into its internal synchronization state.

Consequently, the node becomes permanently desynchronized. It will silently ignore all future legitimate consensus result messages because their true epoch values will never exceed the maliciously inflated `requested_missing_epoch` threshold. The victim node halts consensus participation, failing to track the blockchain head and halting block proposal. This leads to a targeted or network-wide denial of service.

Code Location

Snippet of `process_gossip` function from `crates/consensus/primary/src/network/handler.rs` file:

```
} else {
    let latest_missing = *self.consensus_bus.requested_missing_epoch().borrow();
    if epoch > latest_missing {
        // Not sure we can sanity check this epoch. However if it is bogus the code
        // to handle it should be fine and will reset requested_missing_epoch to
        // sanity.
        self.consensus_bus.requested_missing_epoch().send_replace(epoch);
    }
}
```

 Copy Code

Proof of Concept

Scenario

The following proof of concept code demonstrates how an unauthenticated network payload successfully manipulates the `requested_missing_epoch` metric without provoking an error from the handler.

Test Code

[Copy Code](#)

```
#[tokio::test]
async fn test_crit_02_requested_missing_epoch_manipulation() {
    let TestTypes { handler, consensus_bus, .. } = create_test_types();
    let initial_missing = *consensus_bus.requested_missing_epoch().borrow();

    let attacker_epoch = u32::MAX;
    let mut bogus_result = crate::network::message::ConsensusResult::default();
    bogus_result.epoch = attacker_epoch;
    bogus_result.number = 1;
    bogus_result.hash = BlockHash::repeat_byte(1);

    let gossip = PrimaryGossip::Consensus(Box::new(bogus_result));
    let data = rayls_infrastructure_types::encode(&gossip);
    let topic = TopicHash::from_raw(LibP2pConfig::consensus_output_topic());
    let malicious_msg = GossipMessage { source: None, data, sequence_number: None, topic };

    let res = handler.process_gossip(&malicious_msg).await;

    let manipulated_missing = *consensus_bus.requested_missing_epoch().borrow();
    assert_eq!(manipulated_missing, attacker_epoch);
}
```

Result

[Copy Code](#)

```
Running unittests src/lib.rs (target/debug/deps/rayls_consensus_primary-80ab01dc4b24ea66)

running 1 test
test state_sync::cert_validator::cert_validator_tests::test_poc_v009_signature_bypass ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 68 filtered out; finished in 5.19s

Running tests/it/main.rs (target/debug/deps/it-3c200c95e17371a8)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 16 filtered out; finished in 0.00s
```

BVSS

AO:A/AC:L/AX:L/R:P/S:C/C:N/A:H/I:L/D:N/Y:N (5.1)

Recommendation

It is recommended to ensure that internal synchronization states, particularly `requested_missing_epoch`, are only updated using data retrieved from fully authenticated messages. If the committee for a given epoch is unknown and the signature cannot be verified, the consensus result should fail closed and be quietly ignored or logged, preventing state corruption.

Remediation Comment

SOLVED: The ParFin team solved this issue by unconditionally dropping consensus result messages whose committee is unknown, so that internal synchronization state can only be updated through fully authenticated messages whose BLS signature has been verified against a known committee.

Remediation Hash

<https://github.com/raylsnetwork/axyl/pull/138>

7.4 DOUBLE-VOTING PROTECTION BYPASS DUE TO RACE CONDITION IN REQUEST HANDLER

// MEDIUM

Description

The `RequestHandler` implements a memory-based cache `auth_last_vote` and a database-backed check `read_vote_info` to prevent validators from double-voting (equivocating) in the same round. However, a race condition exists in the vote method where the entry for an authority is removed from the `auth_last_vote` cache before the asynchronous `vote_inner` function is executed.

Because `vote_inner` is an async function that performs database reads and potentially waits for execution results, multiple concurrent vote requests for the same round and authority can bypass both the memory cache and the database check if they arrive before the first request has written its vote to the database. This allows a malicious validator to successfully equivocate by sending multiple different headers for the same round simultaneously, potentially leading to a fork or breaking consensus safety.

Code Location

Snippet of `vote` function from `crates/consensus/primary/src/network/handler.rs` file:

 Copy Code

```
// L364
if let Some((last_epoch, last_round, last_digest, last_response)) =
    auth_last_vote.remove(&header.author)
{
    // ... logic to check for equivocation ...
}
// L427
let res = self.vote_inner(header, parents).await;
// L436
self.auth_last_vote.lock().insert(author, (epoch, round, digest, Some(cached_res)));
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:H/I:H/D:N/Y:N (4.7)

Recommendation

It is recommended to implement a "pending" state or use an entry API pattern that keeps the authority's record in the `auth_last_vote` map throughout the duration of the `vote` request. The entry should only be updated or replaced once the `vote_inner` logic and database write are completed. Furthermore, the database check and write should be performed atomically or protected by a per-authority lock to prevent concurrent requests from bypassing the persistent check.

Remediation Comment

SOLVED: The ParFin team solved this issue by introducing an `AuthVoteState` enum with `InFlight` and `Completed` variants and a dedicated `check_equivocation` function that sets the InFlight marker

atomically under a single lock acquisition before releasing it.

Remediation Hash

<https://github.com/raylsnetwork/axyl/pull/267>

7.5 PLAINTEXT BLS PRIVATE KEY STORAGE IS ALLOWED

// LOW

Description

Node key generation allows storing the BLS private key unencrypted when `passphrase` is not provided. This creates a high-risk default/fallback path where validator signing material is written in recoverable form to disk.

Exploit Scenario

1. Operator runs key generation without passphrase (or automation omits it).
2. Plaintext key file is written to disk.
3. Any actor with read access to that path (local user, backup operator, compromised agent) exfiltrates key.
4. Key is reused to sign as validator.

Impact

- Compromise of host filesystem, backup artifacts, or log/archive leakage can expose validator private key material.
- Attacker with recovered key can impersonate validator signatures and compromise node/consensus trust assumptions.

Code Location

Code of `generate_and_save` function from `crates/infrastructure/config/src/keys.rs` file:

 Copy Code

```
pub fn generate_and_save<RLD: RaylsDirs>(
    rayls_datadir: &RLD,
    passphrase: Option<String>,
) -> eyre::Result<Self> {
    // ...

    if let Some(passphrase) = passphrase {
        let contents = Self::wrap_bls_key(&primary_keypair, &passphrase)?;
        std::fs::write(rayls_datadir.node_keys_path().join(BLS_WRAPPED_KEYFILE), contents)?;
    } else {
        let contents = bs58::encode(primary_keypair.to_bytes()).into_string();
        std::fs::write(rayls_datadir.node_keys_path().join(BLS_KEYFILE), contents)?;
    }
}
```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:M/A:N/I:M/D:N/Y:M (3.8)

Recommendation

It is recommended to:

- Require encrypted key storage by default.
- Gate plaintext mode behind explicit insecure flag (e.g. `--insecure-allow-plaintext-keys`) and disable in production profiles.
- Add startup guard that refuses plaintext keys unless insecure mode is explicitly set.

Remediation Comment

SOLVED: The **ParFin team** solved this issue by making passphrase-based encryption mandatory throughout the entire key lifecycle, eliminating every plaintext storage code path from the codebase.

Remediation Hash

<https://github.com/raylsnetwork/axyl/pull/272>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.